

```

    let head = theString[0];
    let tail = theString.substring(1, theString.length);
    ❸ return rev(tail, head + accum);
  }
}

let text = "abcdef";
document.write("The reverse of " + text + " is " + rev(text) + "<br />");
</script>

```

The conversion of the original recursive functions in *reverseString.py* and *reverseString.html* involves adding an accumulator parameter. The accumulator, named `accum`, is set to the blank string by default if no argument is passed for it ❶. We also change the base case from `return ''` to `return accum` ❷, and the recursive case from `return rev(tail) + head` (which performs a string concatenation after the recursive call returns) to `return rev(tail, head + accum)` ❸. You can think of the `rev('abcdef')` function call as transforming into the following `return`, as shown in [Figure 8-2](#).

By effectively using the accumulator as a local variable shared across function calls, we can make the `rev()` function tail recursive.